

```

import aiosqlite
from datetime import datetime

DB_PATH = "wedding_bot.db"

async def init_db():
    async with aiosqlite.connect(DB_PATH) as db:
        await db.execute("""
        CREATE TABLE IF NOT EXISTS guests (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            telegram_user_id INTEGER UNIQUE NOT NULL,
            name TEXT NOT NULL,
            username TEXT,
            created_at TEXT NOT NULL
        )
        """)
        await db.execute("""
        CREATE TABLE IF NOT EXISTS media (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            guest_id INTEGER NOT NULL,
            telegram_user_id INTEGER NOT NULL,
            file_id TEXT NOT NULL,
            file_unique_id TEXT NOT NULL,
            file_type TEXT NOT NULL,
            backup_chat_id TEXT,
            backup_message_id INTEGER,
            created_at TEXT NOT NULL,
            FOREIGN KEY (guest_id) REFERENCES guests (id)
        )
        """)
        await db.commit()

async def get_guest(telegram_user_id: int):
    async with aiosqlite.connect(DB_PATH) as db:
        db.row_factory = aiosqlite.Row
        async with db.execute(
            "SELECT * FROM guests WHERE telegram_user_id = ?", (telegram_user_id,
        ) as cursor:
            return await cursor.fetchone()

async def create_guest(telegram_user_id: int, name: str, username):

```

```

    async with aiosqlite.connect(DB_PATH) as db:
        await db.execute(
            "INSERT INTO guests (telegram_user_id, name, username, created_at) VA
            (telegram_user_id, name, username, datetime.utcnow().isoformat()),
        )
        await db.commit()
    return await get_guest(telegram_user_id)

async def save_media(guest_id, telegram_user_id, file_id, file_unique_id, file_ty
                    backup_chat_id=None, backup_message_id=None):
    async with aiosqlite.connect(DB_PATH) as db:
        await db.execute(
            """INSERT INTO media
            (guest_id, telegram_user_id, file_id, file_unique_id, file_type,
            backup_chat_id, backup_message_id, created_at)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)""",
            (guest_id, telegram_user_id, file_id, file_unique_id, file_type,
            backup_chat_id, backup_message_id, datetime.utcnow().isoformat()),
        )
        await db.commit()

async def count_media_for_guest(guest_id):
    async with aiosqlite.connect(DB_PATH) as db:
        async with db.execute(
            "SELECT COUNT(*) FROM media WHERE guest_id = ?", (guest_id,)
        ) as cursor:
            row = await cursor.fetchone()
            return row[0]

async def stats():
    async with aiosqlite.connect(DB_PATH) as db:
        async with db.execute("SELECT COUNT(*) FROM guests") as c1:
            guests_count = (await c1.fetchone())[0]
        async with db.execute("SELECT COUNT(*) FROM media") as c2:
            media_count = (await c2.fetchone())[0]
    return guests_count, media_count

async def all_media():
    """Для экспорта: список всех загруженных файлов вместе с именем гостя."""
    async with aiosqlite.connect(DB_PATH) as db:
        db.row_factory = aiosqlite.Row
        async with db.execute("""
            SELECT m.id, g.name, g.username, m.file_type, m.file_id,

```

```
        m.backup_chat_id, m.backup_message_id, m.created_at
FROM media m
JOIN guests g ON g.id = m.guest_id
ORDER BY m.created_at
""" ) as cursor:
    return await cursor.fetchall()
```